

```
from google.colab import files
uploaded = files.upload()
```

Choose Files

Toyota (2).csv

**Toyota (2).csv**(text/csv) - 65629 bytes, last modified: 3/3/2026 - 100% done  
Saving Toyota (2).csv to Toyota (2).csv

```
import pandas as pd
df=pd.read_csv('/content/Toyota (2).csv')
df
```

|      | Unnamed: 0 | Price | Age  | KM    | FuelType | HP  | MetColor | Automatic | CC   | Doors | Weight |  |
|------|------------|-------|------|-------|----------|-----|----------|-----------|------|-------|--------|--|
| 0    | 0          | 13500 | 23.0 | 46986 | Diesel   | 90  | 1.0      | 0         | 2000 | three | 1165   |  |
| 1    | 1          | 13750 | 23.0 | 72937 | Diesel   | 90  | 1.0      | 0         | 2000 | 3     | 1165   |  |
| 2    | 2          | 13950 | 24.0 | 41711 | Diesel   | 90  | NaN      | 0         | 2000 | 3     | 1165   |  |
| 3    | 3          | 14950 | 26.0 | 48000 | Diesel   | 90  | 0.0      | 0         | 2000 | 3     | 1165   |  |
| 4    | 4          | 13750 | 30.0 | 38500 | Diesel   | 90  | 0.0      | 0         | 2000 | 3     | 1170   |  |
| ...  | ...        | ...   | ...  | ...   | ...      | ... | ...      | ...       | ...  | ...   | ...    |  |
| 1431 | 1431       | 7500  | NaN  | 20544 | Petrol   | 86  | 1.0      | 0         | 1300 | 3     | 1025   |  |
| 1432 | 1432       | 10845 | 72.0 | ??    | Petrol   | 86  | 0.0      | 0         | 1300 | 3     | 1015   |  |
| 1433 | 1433       | 8500  | NaN  | 17016 | Petrol   | 86  | 0.0      | 0         | 1300 | 3     | 1015   |  |
| 1434 | 1434       | 7250  | 70.0 | ??    | NaN      | 86  | 1.0      | 0         | 1300 | 3     | 1015   |  |
| 1435 | 1435       | 6950  | 76.0 | 1     | Petrol   | 110 | 0.0      | 0         | 1600 | 5     | 1114   |  |

1436 rows × 11 columns

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
print(df.head())
df.describe()
```

|   | Unnamed: 0 | Price | Age  | KM    | FuelType | HP | MetColor | Automatic | CC   | \ |
|---|------------|-------|------|-------|----------|----|----------|-----------|------|---|
| 0 | 0          | 13500 | 23.0 | 46986 | Diesel   | 90 | 1.0      | 0         | 2000 |   |
| 1 | 1          | 13750 | 23.0 | 72937 | Diesel   | 90 | 1.0      | 0         | 2000 |   |
| 2 | 2          | 13950 | 24.0 | 41711 | Diesel   | 90 | NaN      | 0         | 2000 |   |
| 3 | 3          | 14950 | 26.0 | 48000 | Diesel   | 90 | 0.0      | 0         | 2000 |   |
| 4 | 4          | 13750 | 30.0 | 38500 | Diesel   | 90 | 0.0      | 0         | 2000 |   |

|   | Doors | Weight |
|---|-------|--------|
| 0 | three | 1165   |
| 1 | 3     | 1165   |
| 2 | 3     | 1165   |
| 3 | 3     | 1165   |
| 4 | 3     | 1170   |

|       | Unnamed: 0  | Price        | Age         | MetColor    | Automatic   | CC          | Weight      |
|-------|-------------|--------------|-------------|-------------|-------------|-------------|-------------|
| count | 1436.000000 | 1436.000000  | 1336.000000 | 1286.000000 | 1436.000000 | 1436.000000 | 1436.000000 |
| mean  | 717.500000  | 10730.824513 | 55.672156   | 0.674961    | 0.055710    | 1566.827994 | 1072.45961  |
| std   | 414.681806  | 3626.964585  | 18.589804   | 0.468572    | 0.229441    | 187.182436  | 52.64112    |
| min   | 0.000000    | 4350.000000  | 1.000000    | 0.000000    | 0.000000    | 1300.000000 | 1000.000000 |
| 25%   | 358.750000  | 8450.000000  | 43.000000   | 0.000000    | 0.000000    | 1400.000000 | 1040.000000 |
| 50%   | 717.500000  | 9900.000000  | 60.000000   | 1.000000    | 0.000000    | 1600.000000 | 1070.000000 |
| 75%   | 1076.250000 | 11950.000000 | 70.000000   | 1.000000    | 0.000000    | 1600.000000 | 1085.000000 |
| max   | 1435.000000 | 32500.000000 | 80.000000   | 1.000000    | 1.000000    | 2000.000000 | 1615.000000 |

```
print(df.dtypes)
```

```
Unnamed: 0      int64
Price           int64
Age             float64
KM              object
FuelType        object
HP              object
MetColor        float64
Automatic        int64
CC              int64
Doors           object
Weight          int64
```

```
dtype: object
```

```
df.isnull().sum()
```

|                   | 0   |
|-------------------|-----|
| <b>Unnamed: 0</b> | 0   |
| <b>Price</b>      | 0   |
| <b>Age</b>        | 100 |
| <b>KM</b>         | 0   |
| <b>FuelType</b>   | 100 |
| <b>HP</b>         | 0   |
| <b>MetColor</b>   | 150 |
| <b>Automatic</b>  | 0   |
| <b>CC</b>         | 0   |
| <b>Doors</b>      | 0   |
| <b>Weight</b>     | 0   |

```
dtype: int64
```

```
import numpy as np
num_cols = df.select_dtypes(include=np.number).columns

df[num_cols] = df[num_cols].fillna(df[num_cols].mean())
```

```
import numpy as np

df['KM'] = pd.to_numeric(df['KM'], errors='coerce')
df['HP'] = pd.to_numeric(df['HP'], errors='coerce')

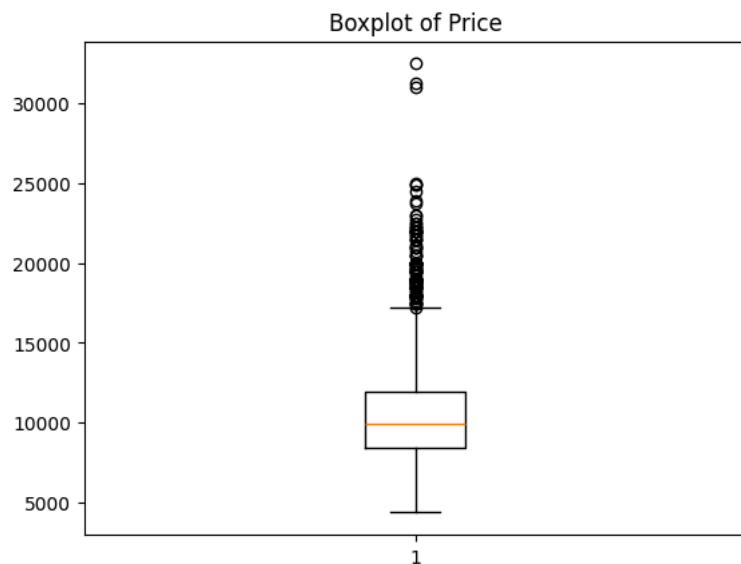
df['Age'] = df['Age'].fillna(df['Age'].mean())
df['KM'] = df['KM'].fillna(df['KM'].mean())
df['HP'] = df['HP'].fillna(df['HP'].mean())
df['Price'] = df['Price'].fillna(df['Price'].mean())
```

```
df['Age'] = df['Age'].fillna(df['Age'].median())
df['KM'] = pd.to_numeric(df['KM'], errors='coerce')
df['KM'] = df['KM'].fillna(df['KM'].median())
```

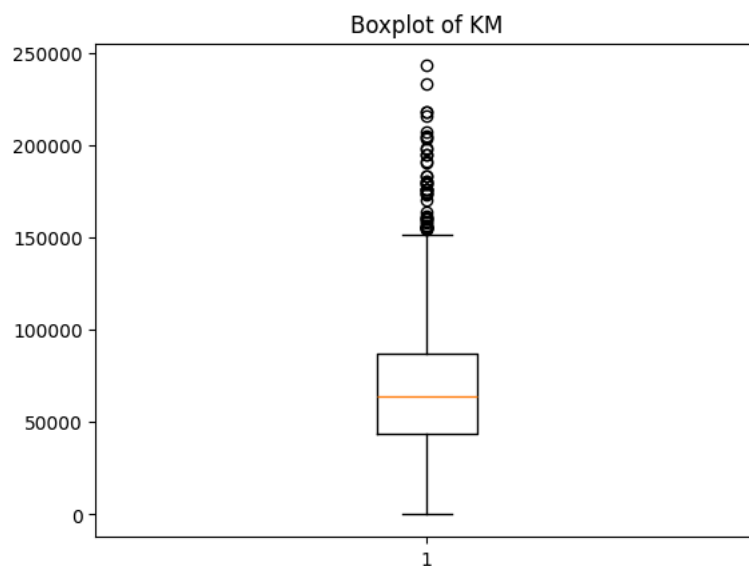
```
df['FuelType'] = df['FuelType'].fillna(df['FuelType'].mode()[0])
df['MetColor'] = df['MetColor'].fillna(df['MetColor'].mode()[0])
```

```
import matplotlib.pyplot as plt

plt.boxplot(df['Price'])
plt.title("Boxplot of Price")
plt.show()
```



```
plt.boxplot(df['KM'])
plt.title("Boxplot of KM")
plt.show()
```



```
from scipy.stats import zscore

z_scores = np.abs(zscore(df[['Price', 'KM']]))
outliers_z = (z_scores > 3).any(axis=1)

print("Number of outliers (Z-score):", outliers_z.sum())
```

Number of outliers (Z-score): 45

```
from scipy.stats import zscore

z_scores = np.abs(zscore(df[['Price', 'KM']]))
outliers_z = (z_scores > 3).any(axis=1)

print("Number of outliers (Z-score):", outliers_z.sum())
```

Number of outliers (Z-score): 45

Explain how these outliers can affect model performance.

Ans : Outliers can mislead the model because they are very different from most data points. They can pull predictions in the wrong direction, lower accuracy, and make the model less reliable. Handling outliers helps the model learn better and give more accurate results.

(d) Encoding Categorical Variables

Identify categorical attributes in the dataset. Ans: : The categorical attributes in the Toyota dataset are FuelType, Color, and Model. These variables represent categories rather than numerical values and must be encoded before applying machine learning algorithms.

(iii) IQR Method

```
Q1 = df[['Price', 'KM']].quantile(0.25)
Q3 = df[['Price', 'KM']].quantile(0.75)
IQR = Q3 - Q1

outliers_iqr = ((df[['Price', 'KM']] < (Q1 - 1.5 * IQR)) |
                (df[['Price', 'KM']] > (Q3 + 1.5 * IQR))).any(axis=1)

print("Number of outliers (IQR):", outliers_iqr.sum())
```

Number of outliers (IQR): 158

Apply: (i) Label Encoding

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df['FuelType_LE'] = le.fit_transform(df['FuelType'])
```

(ii) One-Hot Encoding

```
cat_cols = ['FuelType', 'Doors']
df_encoded = pd.get_dummies(df, columns=cat_cols, drop_first=True)
```

(e) Feature Scaling

- Identify features that require scaling.

Apply:

(i) Normalization (Min-Max Scaling)

```
from sklearn.preprocessing import MinMaxScaler
scaler_minmax = MinMaxScaler()
df_minmax = df_encoded.copy()
df_minmax[num_cols] = scaler_minmax.fit_transform(df_minmax[num_cols])
df_minmax.head()
```

|   | Unnamed: 0 | Price    | Age      | KM      | HP   | MetColor | Automatic | CC  | Weight   | FuelType_LE | FuelType_Diesel | FuelType_Petrol |
|---|------------|----------|----------|---------|------|----------|-----------|-----|----------|-------------|-----------------|-----------------|
| 0 | 0.000000   | 0.325044 | 0.278481 | 46986.0 | 90.0 | 1.000000 | 0.0       | 1.0 | 0.268293 | 1           | True            | False           |
| 1 | 0.000697   | 0.333925 | 0.278481 | 72937.0 | 90.0 | 1.000000 | 0.0       | 1.0 | 0.268293 | 1           | True            | False           |
| 2 | 0.001394   | 0.341030 | 0.291139 | 41711.0 | 90.0 | 0.674961 | 0.0       | 1.0 | 0.268293 | 1           | True            | False           |
| 3 | 0.002091   | 0.376554 | 0.316456 | 48000.0 | 90.0 | 0.000000 | 0.0       | 1.0 | 0.268293 | 1           | True            | False           |
| 4 | 0.002787   | 0.333925 | 0.367089 | 38500.0 | 90.0 | 0.000000 | 0.0       | 1.0 | 0.276423 | 1           | True            | False           |

Next steps: [Generate code with df\\_minmax](#) [New interactive sheet](#)

(ii) Standardization (Z-score scaling)

```
from sklearn.preprocessing import StandardScaler
scaler_standard = StandardScaler()
df_standard = df_encoded.copy()
df_standard[num_cols] = scaler_standard.fit_transform(df_standard[num_cols])
df_standard.head()
```

|   | Unnamed:<br>0 | Price    | Age       | KM      | HP   | MetColor | Automatic | CC       | Weight   | FuelType_LE | FuelType_Diesel | FuelType |
|---|---------------|----------|-----------|---------|------|----------|-----------|----------|----------|-------------|-----------------|----------|
| 0 | -1.730845     | 0.763763 | -1.822802 | 46986.0 | 90.0 | 0.733306 | -0.242893 | 2.314976 | 1.758561 | 1           |                 | True     |